1. INTRODUCTION

Preamble

Ride-hailing services operate in an environment where passenger requests vary over time and across geographic locations. Urban mobility demand is affected by recurring daily routines, weekday and weekend behaviour, weather conditions, public gatherings and local activity centres. A system that only reports historical totals may not provide sufficiently timely information for operational decisions. This project studies a real-time, zone-level demand prediction pipeline for Bengaluru by combining streaming data ingestion, short-window aggregation and machine-learning-based forecasting.

Bengaluru contains employment districts, commercial areas, transit hubs, residential neighbourhoods and event destinations. Demand is therefore not expected to be spatially uniform. A high-demand zone at one time of day may not remain the highest-demand zone at another time. Concerts, festivals, public events or sudden weather changes can create local increases that differ from the normal daily pattern.

The project uses a synthetic Bengaluru ride-hailing dataset containing individual ride-request records. Each record represents one request and includes a timestamp, pickup and drop zones, vehicle type, trip distance, weather, rain intensity and an event indicator. The data is intended to reproduce plausible temporal and spatial mobility patterns for experimentation. Since the records are simulated, findings describe the designed dataset and workflow, not measured market statistics for Bengaluru.

1.1 Objective

- To develop a real-time ride-hailing demand prediction system for Bengaluru using Apache Kafka as the streaming backbone.
- To analyse temporal and spatial patterns in ride requests and estimate demand at Bengaluru-zone level.
- To prepare a short-term forecasting pipeline that uses recent demand history and contextual attributes to predict demand for the next 15 minutes.
- To integrate streaming ingestion, window-based aggregation, prediction output, persistence and dashboard-oriented monitoring.
- To evaluate prediction quality and stream-processing behaviour using defined metrics when implementation measurements are available.

1.2 Scope

The scope covers synthetic ride-request data for Bengaluru, with more than 50 named zones and a simulated observation period of July–August 2026 as stated in the project presentation. The unit of observation is one ride request. The proposed workflow replays historical records according to their timestamps, publishes them to Kafka, aggregates incoming requests in five-minute tumbling windows, derives zone-level features and generates a 15-minute-ahead

demand estimate. The scope includes data preparation, Kafka integration, model development, storage and visualization. It does not claim live integration with a commercial ride-hailing platform, verified real-world driver availability, or operational driver dispatch.

This report documents the project design and EDA evidence shown in the Phase I presentation. Quantitative model metrics, Kafka throughput, end-to-end latency, database measurements and driver-repositioning outcomes were not supplied and must be added after the corresponding experiments are executed.

2. PROBLEM DEFINITION

Preamble

Ride-hailing demand is a spatiotemporal forecasting problem. Requests arrive continuously, and their distribution changes by zone and time interval. The operational challenge is to estimate near-future demand early enough to support monitoring and resource planning. This chapter defines the problem and relates the proposed work to research directions summarized in the project presentation.

2.1 Problem Statement

A ride-hailing operator must respond to an evolving stream of passenger requests distributed across multiple urban zones. If demand increases sharply in a particular zone, a retrospective report may identify the increase only after it has occurred. This delay can contribute to a mismatch between the locations where passengers request rides and the locations where drivers are available. Demand prediction can reduce this information gap by estimating the number of requests likely to arrive in each zone over a short future horizon.

The prediction task is to estimate zone-level ride demand for the next 15 minutes using recent request activity and available contextual features. Streaming input is organised into five-minute tumbling windows. For each window, the system counts requests by pickup zone and constructs features such as recent demand, lagged demand, time of day, day of week, weather, rain intensity and event indicator. The model then produces a forecast for each zone and publishes the prediction as a separate stream.

The problem has interacting dimensions. Temporal variation means that demand changes between early morning, working hours and evening periods. Spatial variation means that commercial areas, transport hubs and residential zones have different demand profiles. Contextual variation means that weather and special events may alter the usual pattern. Finally, a real-time system must process incoming events continuously rather than relying only on static batch computation.

The project therefore addresses both a data-science problem and a data-engineering problem. The data-science component concerns feature construction, forecasting and evaluation. The data-engineering component concerns event ingestion, stream processing, windowing, message delivery, persistence and visualization. A valid demonstration should show that these components work together and that predictions use only data available at prediction time.

Problem Statement: To design and evaluate a Kafka-based real-time system that processes ride-request events, aggregates demand in five-minute windows and predicts zone-level ride-hailing demand 15 minutes ahead for Bengaluru using recent temporal, spatial and contextual information.

2.2 Literature Review

The literature summary in the Phase I presentation indicates that ride-hailing demand forecasting has developed from conventional statistical and machine-learning approaches toward deep spatiotemporal models. Earlier work has explored regression, tree-based methods and neighbourhood-based learning to model demand from historical counts and contextual attributes. Such approaches provide useful baselines because they are comparatively interpretable and can be evaluated with standard forecasting metrics.

More recent studies listed in the presentation consider recurrent neural networks, attention mechanisms, convolutional structures and graph-based methods. These approaches are motivated by the fact that demand in one zone may be related to demand in nearby or functionally connected zones. Recurrent models represent temporal dependencies, while graph-based models represent relationships between geographic regions. Attention mechanisms can help a model focus on relevant historical intervals or spatial relationships.

The reviewed work also points toward the value of contextual variables. Weather, holidays, events, points of interest and other urban attributes may explain demand changes that are not captured by time-of-day patterns alone. The project presentation identifies research on cross-city prediction, uncertainty quantification, spatiotemporal forecasting, district-centric attention, hybrid deep-learning methods and multimodal demand prediction. These directions support including weather, rain intensity and event indicators in the dataset.

A further distinction is between prediction-model research and end-to-end deployment. A forecasting model may be trained and evaluated offline without addressing how new requests are ingested, grouped into windows, transformed into features and delivered to downstream services. The proposed project focuses on this integration through Kafka event transport, five-minute demand aggregation, a 15-minute prediction stage, output publication, storage and visualization.

The research gap addressed here is practical and architectural: demonstrating a cohesive streaming workflow in which zone-level demand features are computed from an incoming event stream and used to produce short-horizon forecasts. The project does not claim to introduce a new forecasting algorithm. Its contribution is the integration and evaluation of context-aware demand forecasting within a real-time streaming architecture, with event-driven surge scenarios as a potential extension.

The papers named in the presentation are retained in the references section using the bibliographic details visible in the supplied slides. Several entries do not include complete volume, issue, page, DOI or publisher information. Those details must be checked against the original publications before final academic submission; they have not been invented.

3. DATA COLLECTION AND PROCESSING

Preamble

This chapter describes the dataset supplied for the project and the preparation required before records can be used by the streaming and forecasting pipeline. The Phase I presentation identifies the data as a synthetic event-level dataset for Bengaluru. The distinction between synthetic and observed data is important because it determines how findings should be interpreted.

3.1 Dataset Description

The dataset is titled Synthetic Bengaluru Ride-Hailing Dataset. Its geographic scope is Bengaluru, Karnataka, and its stated time period is July–August 2026. It contains individual ride-request events and represents more than 50 Bengaluru zones. One row corresponds to one ride request, rather than one pre-aggregated time interval. This event-level structure is appropriate for replaying records through Kafka and deriving time-window aggregates during stream processing.

The dataset was generated through stochastic simulation intended to reproduce realistic temporal and spatial mobility patterns. The project should document the simulation assumptions, including how weekday/weekend variation, peak-hour intensity, zone popularity, weather effects and event-related demand are generated. Such documentation is necessary for reproducibility and for assessing whether the simulation contains plausible dependencies rather than arbitrary random variation.

The presentation lists many pickup and drop zones and shows substantial differences in request counts across pickup locations. The highest-demand pickup locations in the displayed chart include Indiranagar and Koramangala, followed by Majestic (KBS), KIAL Airport and MG Road. These are findings from the supplied synthetic dataset and should not be presented as verified real-world rankings.

3.2 Data Features and Format

The following fields are visible in the dataset schema presented in the Phase I slides:

- Request_ID: unique identifier for an individual ride request.
- Timestamp: date and time at which the request is recorded; used for chronological ordering and time-based feature extraction.
- Pickup_Zone: geographic zone where the passenger requests pickup; used as the primary spatial key for demand aggregation.
- Drop_Zone: destination zone; useful for origin-destination analysis and understanding movement between zones.
- Vehicle_Type: requested vehicle category; may support segmented demand analysis if category-level forecasts are required.

- Ride_Distance_KM: trip distance in kilometres; a descriptive trip attribute that may support additional analysis.
- Weather: categorical weather condition associated with the request or time interval.
- Rain_Intensity: measure or category representing rainfall intensity.
- Event_Flag: indicator for whether the request is associated with an event-affected period or zone.

For the principal forecasting target, records should be grouped by timestamp window and Pickup_Zone. The target variable is the number of requests in a zone during the future 15-minute horizon. Drop_Zone, Vehicle_Type and Ride_Distance_KM can be retained for supplementary analyses, but their inclusion in the forecasting model should be justified by validation performance and by whether the values are available at prediction time.

3.3 Pre-processing Techniques Applied

The preprocessing stage should validate the schema and data types. Request_ID should be checked for uniqueness, Timestamp should be parsed into a consistent datetime representation, and pickup/drop zone values should be checked against the approved zone list. Numeric fields such as Ride_Distance_KM and Rain_Intensity should be converted to numeric types with invalid values identified rather than silently coerced.

Records should be sorted chronologically before replay. Missing timestamps or invalid zone identifiers should be handled according to a documented rule. Duplicate request identifiers should be investigated because duplicates can inflate demand counts. Categorical values should be normalised so that spelling variations do not create separate categories. The processing pipeline should check for impossible distances, invalid timestamps and records outside the stated simulation period.

Temporal feature preparation can derive hour of day, day of week, weekend indicator and other calendar features from Timestamp. Lag features should be computed using only earlier windows. For example, recent demand and lagged demand for a zone must be calculated without using future observations. Contextual fields such as Weather, Rain_Intensity and Event_Flag should be aligned to the corresponding event or time window.

Finally, the data should be divided chronologically into training, validation and test periods. Random splitting can leak information from later periods into training and may overstate forecasting performance. The same feature-generation logic should be used during offline training and streaming inference so that the model receives consistent inputs.

4. Methodology

Preamble

The proposed methodology combines offline data preparation, timestamp-based stream replay, Kafka message transport, window-based aggregation, feature generation, machine-learning prediction and output persistence. The architecture shown in the supplied Phase I presentation is the basis for this chapter. The five-minute aggregation interval and 15-minute prediction horizon are retained as specified in that architecture

4.1 System Architecture

The system is organised into three logical stages. The first stage prepares historical ride-request data and simulates a live stream by replaying records in timestamp order. The second stage consumes request events from Kafka and performs real-time processing. The third stage generates demand predictions, publishes the predictions to Kafka, stores the outputs and makes them available for dashboard visualisation.

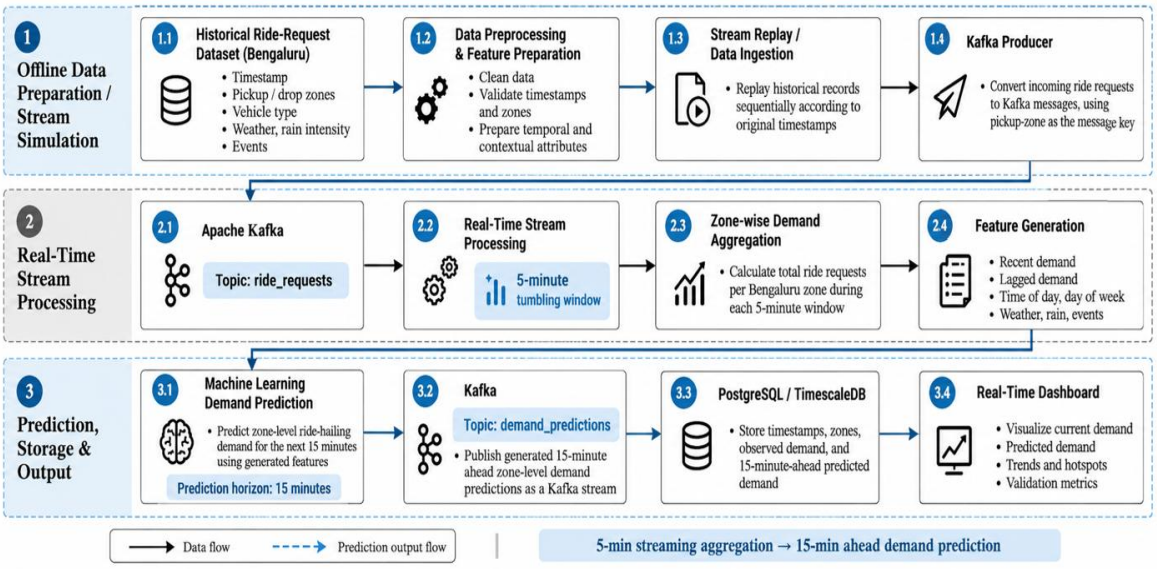


Figure 1. Architecture and methodology of the proposed real-time ride-hailing demand prediction system.

As illustrated in Figure 1, historical ride records are cleaned and validated before replay. A producer converts each ride request into a Kafka message, using the pickup zone as the message key. The ride_requests topic carries incoming events to the stream-processing component. Five-minute tumbling windows calculate request counts for each Bengaluru zone. The resulting aggregates and contextual variables are passed to the machine-learning prediction stage, which estimates demand for the next 15 minutes. Predictions are published to the demand_predictions topic and persisted in PostgreSQL or TimescaleDB for dashboard access.

4.2 Implementation of AI Pipeline

The AI pipeline begins with construction of a training table from historical event records. Events are grouped into five-minute intervals by Pickup_Zone to create observed demand counts. For each zone and interval, candidate predictors include recent demand, lagged demand, hour of day, day of week, weather, rain intensity and event indicator. The model target is the number of ride requests expected in the subsequent 15-minute interval. The exact model family should be selected through validation rather than assumed in advance.

A reasonable baseline is a historical or statistical model using recent demand and calendar features. Tree-based regressors such as Random Forest or gradient-boosted trees can then be compared with the baseline because they can model nonlinear interactions between time, zone and context. If the project later adopts a recurrent or graph-based model, it should be compared against the simpler baseline under the same chronological split and evaluation protocol.

During inference, the streaming component creates the current feature vector for each zone after the five-minute window closes. The trained model receives only features that would have been available at that point in time. It produces a 15-minute-ahead estimate, associated with the zone, forecast creation time and target time interval. The forecast should be stored together with model version and input-window metadata to support traceability.

Model performance should be measured using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and, where useful, a scale-aware metric such as weighted absolute percentage error. Metrics should be reported on a held-out chronological test period and separately for high-demand and low-demand zones where possible. For surge detection, evaluation can additionally measure precision, recall and warning lead time after defining a surge threshold in advance.

4.3 Big Data Integration

Apache Kafka is the event-streaming backbone. The producer publishes individual ride requests to the ride_requests topic. The pickup-zone key allows records for the same zone to be routed consistently to a partition, subject to the configured partitioning strategy. Consumers read the topic and maintain the processing state required for time-window aggregation. Consumer groups can scale processing, while offsets support recovery and replay.

The five-minute tumbling window converts the unbounded event stream into bounded aggregates. For each window and pickup zone, the system computes the total number of ride requests. Tumbling windows are non-overlapping, so each event belongs to one five-minute interval. The aggregation result is combined with time and contextual features before prediction.

The prediction service publishes forecasts to a separate Kafka topic named demand_predictions. Separating input events from prediction outputs makes the workflow

easier to monitor and allows downstream consumers to subscribe without interfering with request ingestion. PostgreSQL or TimescaleDB stores timestamps, zones, observed demand and predicted demand. A dashboard can query this store to show current demand, predicted demand, temporal trends, high-demand zones and model-validation metrics.

Reliability considerations include appropriate message acknowledgement settings, replication where available, consumer offset management, idempotent writes and handling of late or malformed events. The system should record processing timestamps so that end-to-end latency can be measured. Kafka transports and retains messages; zone aggregation, feature generation and forecasting are application-level processing tasks built on top of the stream.

For a reproducible demonstration, the replay producer should preserve event-time ordering or explicitly document how out-of-order records are handled. Replay speed can be controlled to simulate real-time arrival. The report should distinguish event time, when the ride request occurred, from processing time, when the pipeline received and processed the message.

5. Analysis and Results

Preamble

The analysis in this report is based on the exploratory data analysis figures and observations included in the Phase I presentation. These figures describe the synthetic dataset. The presentation does not provide numerical forecasting metrics, Kafka throughput, processing latency or resource-utilisation measurements; accordingly, those results are not fabricated here and are identified as evaluation items for the implementation phase.

5.1 Exploratory Data Analysis

The hourly ride-demand chart in the Phase I presentation shows a non-uniform distribution across the 24 hours of the day. Demand is comparatively low during early morning hours and rises through the day, with a pronounced evening peak between approximately 17:00 and 21:00. The highest bar is shown around 19:00. This pattern supports the use of hour-of-day features and demonstrates why a uniformly distributed synthetic demand profile would be unsuitable for evaluating a demand forecasting system.

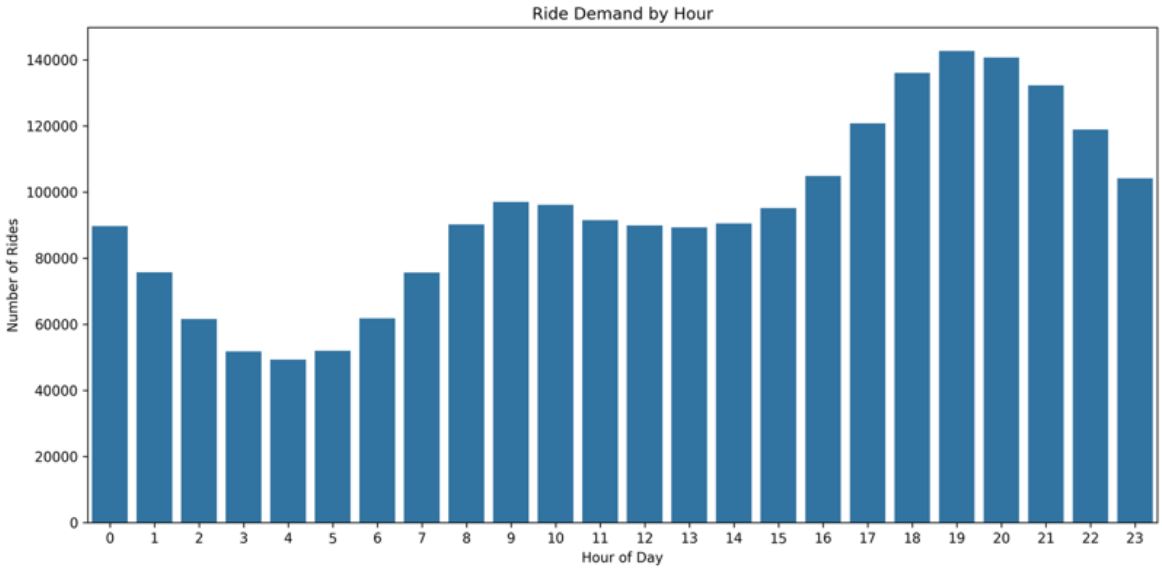


Figure 2. Hourly Distribution of Ride-Hailing Demand in Bengaluru

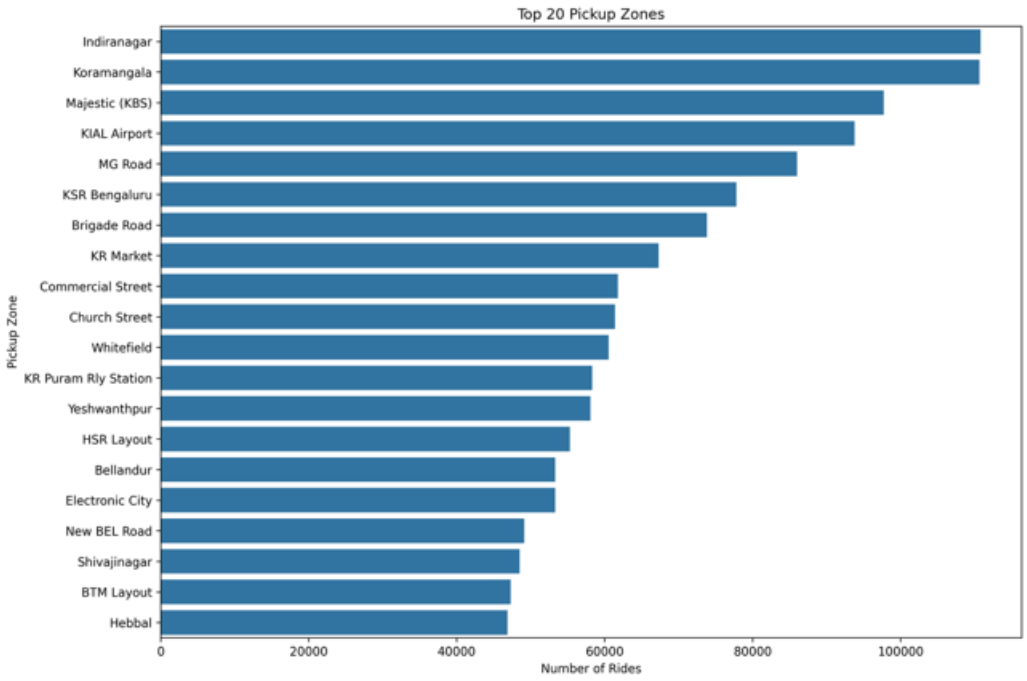


Figure 3. Top 20 Pickup Zones Ranked by Total Ride-Hailing Demand.

The horizontal bar chart in Figure 3 ranks pickup zones by total ride count. Indiranagar and Koramangala appear among the highest-volume zones, with Majestic (KBS), KIAL Airport and MG Road also showing substantial activity. The variation across zones indicates that a city-wide aggregate alone would hide meaningful local differences. Zone-level aggregation is therefore central to the proposed forecasting task.

5.2 Temporal and Spatial Demand Patterns

The Phase I presentation reports that Friday and Saturday show higher overall demand than most weekdays. It also reports stronger evening and night demand in zones such as Indiranagar and Koramangala. These observations suggest that the model should distinguish both hour-of-day and day-of-week effects and should not assume that all zones follow the same demand curve.

The pickup-to-drop heatmap shown in Figure 4 represents the distribution of trips between origin and destination zones. Darker cells indicate larger counts in the visualisation. Stronger activity is visible for several established commercial, transport and activity centres. An origin-destination view can complement pickup-zone demand counts by revealing movement patterns, although the primary forecast target in this project remains pickup demand by zone.

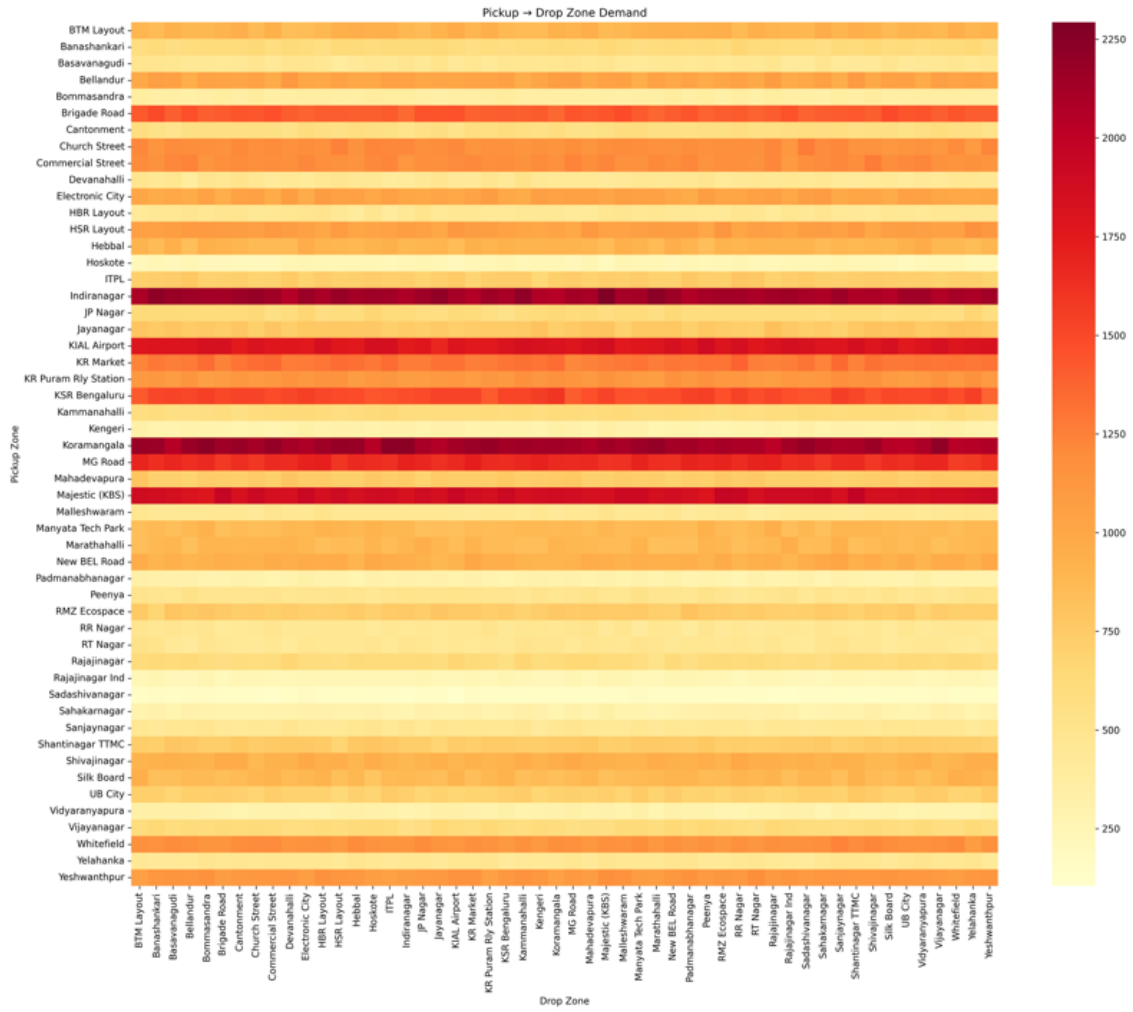


Figure 4. Pickup-to-Drop Zone Ride-Demand Heatmap for the Synthetic Bengaluru Dataset.

The presentation also notes short-term fluctuations in five-minute demand. Such variation motivates window-based stream processing: the system needs to convert individual requests into recent zone-level counts before the prediction stage can operate. The five-minute window is a design choice from the proposed architecture and should be tested against alternative window sizes if time permits.

5.3 Streaming and Prediction Analysis

The planned streaming analysis examines whether each request can be ingested, assigned to the correct event-time window, included in the appropriate zone aggregate and made available to the prediction stage. A five-minute tumbling window produces a recent demand snapshot for each pickup zone. The model then estimates the next 15 minutes. These intervals serve different purposes: five minutes is the observation/aggregation interval, while 15 minutes is the forecast horizon.

The system should display observed demand and predicted demand side by side. A useful demonstration is to select a zone, show the requests accumulated during the latest five-minute window, and then show the model's estimate for the following 15 minutes. A second view can rank zones by forecast demand so that emerging hotspots are visible. Event and weather indicators can be toggled or compared in controlled scenarios to examine whether they improve prediction.

An event-aware experiment can compare a baseline model using recent demand and calendar features with a context-aware model that also uses weather, rain intensity and Event_Flag. The models should be trained and tested on identical time splits. If the synthetic data generator explicitly increases demand around events, the test should verify that the feature is available at inference time and that the evaluation period includes event and non-event examples.

5.4 Performance Evaluation Plan

Forecast accuracy should be reported using MAE and RMSE for the held-out test period. Results should be computed across all zones and, where possible, reported by zone or demand category. A simple baseline is essential: the proposed model should demonstrate measurable improvement over a persistence forecast or historical average before a more complex model is justified.

Streaming performance should be measured independently of prediction accuracy. Relevant measures include records processed per second, average and percentile processing latency, consumer lag, window completion delay, prediction publication delay and database write latency. Reliability tests should include producer restart, consumer restart and replay from committed offsets. Results should state hardware, software versions, Kafka configuration, number of partitions and replay rate.

5.5 Limitations and Validity

The dataset is synthetic and therefore cannot establish actual Bengaluru ride-hailing demand or prove operational effectiveness. The quality of the experiment depends on the realism of the simulation assumptions. If event flags, rain conditions and ride counts are generated with overly simple or deterministic rules, the model may learn those rules rather than generalisable mobility behaviour.

The EDA figures are reproduced from the supplied presentation and are not recalculated from the raw CSV in this task. Exact dataset row count, missing-value percentages, duplicate count and statistical summaries were not available in the supplied PPT. These values should be added after running the final EDA script against the exact dataset version used by the pipeline.

The presentation describes PostgreSQL/TimescaleDB as the storage layer and a real-time dashboard as the output component. It does not show measured database performance or screenshots of a completed dashboard. Similarly, the architecture specifies a 15-minute prediction horizon but does not report model accuracy. These are implementation and evaluation tasks, not completed results established by the supplied material.

5.6 Interpretation and Practical Value

The practical value is advance identification of zones where demand may rise. If the forecast is sufficiently accurate and timely, an operations team could use the information to monitor potential hotspots and consider driver repositioning. This report treats repositioning as a decision-support use case, not as an implemented dispatch algorithm, because driver locations, availability, travel time and repositioning constraints are not included in the stated dataset schema.

A useful extension is to evaluate demand surges around concerts, festivals, sporting events and sudden weather changes. The system can compare expected demand under normal conditions with the forecast under an event or weather scenario. This can help demonstrate whether contextual signals add value beyond recurring rush-hour patterns. The value should be quantified through forecast error and surge-warning lead time rather than asserted from visual inspection alone.

6. Conclusion

This project proposes a real-time, zone-level ride-hailing demand prediction system for Bengaluru. The supplied synthetic dataset and EDA indicate clear temporal and spatial variation, including an evening demand peak, weekday/weekend differences and higher request volumes in selected pickup zones. These patterns support short-window aggregation and zone-specific forecasting.

The proposed architecture uses Kafka to ingest ride-request events, five-minute tumbling windows to calculate recent zone-level demand, a machine-learning stage to forecast demand 15 minutes ahead, a separate Kafka topic for prediction outputs, and PostgreSQL/TimescaleDB for persistence and dashboard access. The project combines a data-science objective—short-term demand estimation—with a streaming-engineering objective—continuous, reliable event processing.

The current evidence establishes the dataset design, EDA observations and system architecture. It does not yet establish model accuracy, streaming throughput, latency or real-world operational impact. The next phase should implement the end-to-end pipeline, compare a baseline and context-aware forecasting model, measure stream-processing performance, and evaluate event-driven surge scenarios. Because the data is synthetic, conclusions should remain limited to controlled experimentation until validated against real ride-request data.

7. References

[1] Chi et al., “Zero-shot Cross-city Ride-hailing Demand Prediction,” Transportation Research Part A, 2026.

[2] Liu et al., “Uncertainty Quantification for Joint Demand Prediction of Multi-modal Ride-sourcing Services,” Transportation Research Part C, 2026.

[3] Zhang and Guo, “Ride-hailing Spatiotemporal Demand Forecasting via LLM-driven Semantic Quantification,” Journal of Computer Applications, 2026.

[4] Wang and Fu, “Spatiotemporal Multiconvolution and Attention Mechanisms for Ride-hailing Demand Forecasting,” Journal of Control Science and Engineering, 2025.

[5] Roy et al., “Spatial Transferability of Machine Learning Based Models for Ride-hailing Demand Prediction,” Transportation Research Part A, 2025.

[6] Shang and Li, “Security-Enhanced Spatiotemporal Ride-Hailing Demand Prediction,” IEEE Transactions on Intelligent Transportation Systems, 2025.

[7] Wen et al., “Spatial-Temporal Hierarchical Modeling for Multi-step Ride-hailing Demand Forecasting,” International Journal of General Systems, 2025.

[8] Eslami and Gh, “District-Centric Attention Mechanism for Online Ride-hailing Demand Forecasting,” IEEE Access, 2024.

[9] Li et al., “Demand Forecasting and Predictability Identification of Ride-Sourcing,” Transportation Research Part C, 2024.

[10] Zhong et al., “Short-Term Origin-Destination Demand Prediction,” Transportation Research Record, 2024.

[11] Wang et al., “Ride-hailing OD Demand Prediction with Spatiotemporal Information Fusion,” Transportation Science and Engineering, 2024.

[12] Zheng et al., “Encoder Framework for Taxi-Demand Prediction,” Engineering Applications of Artificial Intelligence, 2023.

[13] Wang et al., “RF-BiLSTM with Attention for Online Ride-Hailing Demand Forecasting,” Symmetry, 2023.

[14] Li et al., “Hybrid Deep Learning Models for Short-Term Online Car-Hailing Demand,” Transportation Letters, 2023.

[15] Chen et al., “H-ConvLSTM-based Bagging Learning for Ride-Hailing Demand,” Transportation Research Part C, 2022.

[16] Wang et al., "Dynamic Multi-Graph Convolutional Network for OD-based Ride-Hailing Demand," Information Sciences, 2022.

[17] Chen et al., "Short-Term Prediction of Demand for Ride-Hailing Services," Data Science for Transportation, 2021.

[18] Lu et al., "Short-Term Demand Forecasting for Online Car-Hailing," Physica A, 2021.

[19] Ke et al., "Joint Prediction of Multi-modal Ride-Hailing Demands," Transportation Research Part C, 2021.

[20] Rahman et al., International Journal of Transportation Science and Technology, 2021.

[21] Huang et al., IEEE Transactions on Intelligent Transportation Systems, 2021.

[22] Jin et al., "Urban Ride-Hailing Demand Prediction with Multiple Spatiotemporal Information Fusion," Transportation Research Part C, 2020.

[23] Kim et al., "Interpretable ML Framework for City-wide Taxi/FHV Demand Prediction," Transportation Research Part C, 2020.

[24] Hsu and Chen, "Taxi Demand Prediction Using LSTM with Residuals and Multi-Head Attention," Proceedings of ICINCO, 2020.

8. Appendix: Snapshots

Preamble

This appendix contains selected snapshots reproduced from the supplied Phase I presentation. Each figure is referenced in the report text and captioned to identify its content and source. The images document the EDA and proposed architecture; they are not independent validation of model or streaming performance.